naming convention). The reason Compose creates this network for us is simple: it knows that the services we're defining are all related to the same application, so inevitably they are going to need to talk to one another.

But how do containers on this network find each other?

All Docker networks (except for the legacy bridge network) have built-in Domain Name System (DNS) name resolution. That means that we can communicate with other containers running on the same network by name. Compose uses the service name (as defined in our docker-compose.yml) as the DNS entry. So if we wanted to reach our web service, that's accessible via the hostname web. This provides a basic form of *service discovery*—a consistent way of finding container-based services, even across container restarts.

This explains how we were able to connect from the ad-hoc container running the redis-cli to our Redis server running as the redis service. Here's the command we used:

```
$ docker-compose run --rm redis redis-cli -h redis
```

The option -h redis says, "Connect to the host named redis." This only worked because Compose had already created our app's network and set up DNS entries for each service. In particular, our redis service can be referred to by the hostname redis.

Our Rails App Talking to Redis

Although it's great that we've started up a Redis server using Compose, it's not much use to us by itself. The whole point of running the Redis server is so our Rails app can talk to it and use it as a key-value store. So let's connect our Rails app to Redis and actually use it for something. Sound like fun?

Now, there are a million ways an app might want to use Redis. For our purposes, though, we don't really care *what* we use Redis for; we care more about *how* to use it. We're going to use an intentionally basic example: our Rails app will simply store and retrieve a value. However, keep the larger point in mind—once you know how to set up the Rails app to talk to the Redis server in a container, you can use it however you like.

Ready? Let's begin.

Installing the Redis Gem

The first thing we need to do to get our Rails app talking to Redis is to install the redis gem. You may remember that to update our gems, we need to update our image as we saw on page 55.